harness.py 학습 노트 — asyncio 병렬 에이전트 실행

작성일: 2026-06-05 작성자: Dev2 담당

---

1. 한 줄 요약

common/harness.py는 여러 에이전트를 동시에 실행하고, 30초 안에 안 끝나면 빈 결과로 대체하며, 실행 기록을 DB에 자동으로 남기는 "주방장" 모듈이다.

---

2. 4-Layer 요약

Layer 1. 비유 — 식당 주방

두 요리사(정적 분석 / 보안 스캔)가 동시에 요리한다. 한 명이 30초 안에 안 오면 빈 접시(timeout_result)를 대신 낸다. asyncio.gather = 주방장이 "동시 시작" 신호를 보내는 도구. asyncio.wait_for = "30초 이내에 안 오면 빈 접시" 규칙.

Layer 2. 개념

```
Python asyncio 이벤트 루프 (싱글 스레드)
  ├─ Task 1: sa_graph.ainvoke(state)
  └─ Task 2: ss_graph.ainvoke(state)

asyncio.gather(A, B)  → 둘 다 끝나면 [결과A, 결과B]
asyncio.wait_for(코루틴, timeout=30)  → 초과 시 asyncio.TimeoutError
```

핵심: 실제로 동시 실행이 아니라 I/O 대기 시간에 다른 코루틴으로 양보(yield).

Layer 3. 왜 harness가 필요한가

직접 asyncio.gather를 쓰면:

- 타임아웃 처리 코드가 graph.py 곳곳에 중복
- agent_traces INSERT/UPDATE 빠뜨리기 쉬움
- 타임아웃 vs 다른 예외 구분이 불명확

harness를 쓰면:

- run_parallel_agents() 한 줄 호출로 끝
- 타임아웃/트레이스/빈 결과 대체가 자동
- CLAUDE.md §6 "직접 gather 금지" 준수

Layer 4. 실제 코드 핵심

```
# _run_one 내부 — try/except가 async with 바깥에 있어야 함
try:
```

```
    async with AgentHarness(...) as harness:   # __aenter__: trace INSE
        result = await harness.run(fn, *args)  # TimeoutError 전파
    # __aexit__: status='fail' 기록 후 TimeoutError 전파
except asyncio.TimeoutError:
    return make_timeout_result(agent_name)     # 빈 결과 대체
```

---

3. 가장 헷갈리는 부분 — TimeoutError와 aexit

잘못된 패턴 (버그)

```
async with AgentHarness(...) as harness:
    try:
        result = await harness.run(fn, *args)
    except asyncio.TimeoutError:           # ← 여기서 잡으면 안 됨
        return make_timeout_result(name)
# __aexit__(exc_type=None) → status='success' ← 거짓 성공 기록!
```

올바른 패턴

```
try:
    async with AgentHarness(...) as harness:
        result = await harness.run(fn, *args)
    # TimeoutError 발생 시 __aexit__(exc_type=TimeoutError) → status='fa
except asyncio.TimeoutError:               # ← with 블록 밖에서 잡음
    return make_timeout_result(name)
```

이유: `async with`의 `__aexit__`은 블록 안에서 발생한 예외를 인자로 받는다. 안에서 잡으면 `__aexit__`에 `None`이 전달되어 성공으로 처리된다.

---

4. 파일 구조

```
common/
  __init__.py
  harness.py          ← 이 파일
    ├─ make_timeout_result()   빈 Output Contract 생성
    ├─ AgentHarness            컨텍스트 매니저 (trace 기록)
    ├─ _run_one()              에이전트 1개 실행 + 타임아웃 처리
    ├─ run_parallel_agents()   공개 API — graph.py에서 호출
    └─ instrument()            LangSmith 데코레이터 (Dev3과 동일)
```

---

5. 사용 방법

```
from common.harness import run_parallel_agents

results = await run_parallel_agents(
    agents=[
        ("static_analysis", sa_graph.ainvoke, (state,), {}),
        ("security_scan",   ss_graph.ainvoke, (state,), {}),
```

```
    ],
    db_pool=pool,           # asyncpg 커넥션 풀 (None이면 DB 기록 생략)
    pr_trace_id=pr_trace,  # 부모 trace ID
    task_id=state["task_id"],
)

# results[0] = static_analysis 결과 (또는 타임아웃 시 빈 dict)
# results[1] = security_scan 결과 (또는 타임아웃 시 빈 dict)

# instrument 데코레이터 사용
from common.harness import instrument

@instrument(name="bandit_scan", run_type="tool")
async def run_bandit(diff: str) -> dict:
    ...
```

---

6. 연관 규칙 (CLAUDE.md)

섹션	규칙
§3	agent_traces 로깅 — 모든 에이전트 시작/종료 기록
§6	타임아웃 30초, run_parallel_agents() 사용 필수
§10-4	에러는 삼키지 않는다 — 타임아웃 외 예외는 전파

---

7. 핵심 개념 한 줄씩

- `asyncio.gather` — 여러 코루틴을 한 번에 예약하고 모두 끝날 때까지 기다림
- `asyncio.wait_for` — 하나의 코루틴에 타임아웃 적용
- `async with` (`__aenter__`/`__aexit__`) — 진입/종료 시 자동 실행되는 코드 블록
- `return False` in `__aexit__` — 예외를 억제하지 않음, 상위로 전파
- `secrets.token_hex(2)` — 4자리 랜덤 hex (충돌 없는 서브 trace ID 생성용)